

Sujets de Recherche en Intelligence Artificielle Agentique

LLM · Agents · MCP · A2A · RAG · Mémoire · Orchestration

15 sujets publiables — Niveau Master / Ingénieur / Doctorat

Spécialité : IA Agentique

Avril 2026

Sujet 1 — Évaluation comparative des frameworks d'agents IA : LangChain, CrewAI, AutoGen et LlamaIndex

Problématique

Les agents IA reposent aujourd'hui sur des frameworks hétérogènes (LangChain, CrewAI, AutoGen, LlamaIndex) qui diffèrent considérablement dans leurs paradigmes d'orchestration, leurs mécanismes d'appel d'outils et leurs performances. L'absence de benchmarks standardisés et reproductibles empêche les praticiens et chercheurs de choisir objectivement le framework le plus adapté à leur contexte applicatif. Cette recherche vise à combler ce vide en proposant une méthodologie d'évaluation systématique couvrant la latence, la fiabilité, la complétude des tâches et la consommation de ressources.

Mots-clés

Agent benchmarking, Tool calling, ReAct pattern, LangGraph, CrewAI, AutoGen, LlamaIndex, orchestration

Abstract

The proliferation of agentic AI frameworks has created an urgent need for rigorous comparative evaluation methodologies. This paper presents a systematic benchmark study of four leading agentic frameworks — LangChain, CrewAI, AutoGen, and LlamaIndex — across standardized task suites involving tool use, multi-step reasoning, and collaborative agent execution. We introduce AgentBench-FR, an open evaluation suite comprising 120 tasks across five difficulty levels, and measure framework performance along four dimensions: task completion rate, average latency, resource consumption, and error resilience. Our results reveal significant performance disparities: AutoGen achieves the highest completion rate on multi-agent tasks (83.4%) while LangGraph demonstrates superior latency in single-agent pipelines. We further analyze failure modes and propose architectural guidelines for selecting the appropriate framework

based on application constraints. Our benchmark suite is publicly released to foster reproducible research in agentic AI.

Sujet 2 — Model Context Protocol (MCP) : standardisation de l'accès aux outils pour les grands modèles de langage

Problématique

Les grands modèles de langage (LLM) interagissent avec des outils externes via des interfaces propriétaires et non interopérables, créant des problèmes de portabilité, de sécurité et de maintenabilité. Le Model Context Protocol (MCP), introduit par Anthropic en 2024, propose un standard ouvert pour l'exposition d'outils aux LLM. Cependant, son adoption soulève des questions fondamentales sur sa sécurité face aux attaques par injection de contexte, son passage à l'échelle dans des environnements distribués, et sa capacité à gérer des outils temps-réel critiques. Cette recherche analyse l'architecture MCP, évalue ses limites et propose des extensions pour des cas d'usage industriels exigeants.

Mots-clés

MCP, Model Context Protocol, Tool use, API standardisation, Anthropic, SSE transport, LLM tools, interoperability

Abstract

The Model Context Protocol (MCP), introduced by Anthropic in late 2024, represents a significant step toward standardizing how large language models discover and invoke external tools. This paper provides the first comprehensive security and scalability analysis of MCP in production environments. We examine MCP's transport layer (SSE and stdio), its tool discovery mechanism, and its permission model against a threat taxonomy of six attack vectors, including tool poisoning, context injection, and unauthorized capability escalation. Through empirical evaluation on a testbed of 30 MCP servers spanning database, code execution, and web browsing tools, we quantify throughput, latency, and security posture. Our findings reveal critical vulnerabilities in tool metadata validation and propose MCP-Secure, an extension layer adding cryptographic attestation, rate limiting, and sandboxed execution contexts. We argue that MCP, augmented with our proposed safeguards, can serve as a universal, secure substrate for LLM-tool integration in enterprise settings.

Sujet 3 — Architectures mémoire long-terme pour agents IA : mémoire épisodique, sémantique et procédurale

Problématique

La fenêtre de contexte limitée des LLM constitue un frein majeur au déploiement d'agents IA dans des applications nécessitant une continuité conversationnelle et un apprentissage cumulatif. Bien que plusieurs approches de mémoire externe existent (bases vectorielles, graphes de connaissance, mémoire compressive), aucune architecture unifiée ne combine efficacement les trois types de mémoire cognitifs : épisodique (événements passés), sémantique (connaissances structurées) et procédurale (savoir-faire). Cette recherche propose et évalue un cadre d'architecture mémoire hybride permettant aux agents de maintenir une cohérence identitaire et factuelle sur des sessions multiples.

Mots-clés

Vector database, Knowledge graph, MemGPT, Letta, Episodic memory, Semantic memory, Long-term agent memory, Memory compression

Abstract

Persistent memory is a fundamental capability gap in current agentic AI systems. While large language models excel within their context window, they lack mechanisms for retaining and retrieving information across sessions, limiting their utility in longitudinal applications. This paper introduces TriMem, a hybrid memory architecture that integrates episodic memory (interaction logs with temporal indexing), semantic memory (structured knowledge graphs with entity resolution), and procedural memory (reusable skill representations) into a unified retrieval framework. TriMem employs a dynamic memory consolidation algorithm that compresses episodic traces into semantic representations, reducing storage overhead by 67% while maintaining factual accuracy. We evaluate TriMem against MemGPT, Zep, and naive RAG baselines on three longitudinal benchmarks: a 30-session customer support simulation, a multi-week project management scenario, and a long-horizon research assistant task. TriMem achieves a 41% improvement in cross-session coherence and a 28% reduction in factual contradictions compared to the best baseline. Our architecture provides a principled foundation for building agents with genuine long-term memory.

Sujet 4 — RAG multimodal : récupération augmentée par la génération sur corpus hétérogènes texte, image et audio

Problématique

Les systèmes de Retrieval-Augmented Generation (RAG) sont largement déployés pour des corpus textuels, mais échouent à traiter efficacement des documents industriels réels qui combinent texte, tableaux, schémas, diagrammes et annotations manuscrites. La conversion OCR introduit des pertes d'information structurelle significatives, tandis que les embeddings monomodaux ne capturent pas les relations sémantiques entre éléments visuels et textuels. Cette recherche conçoit et évalue un pipeline RAG multimodal exploitant des modèles d'embeddings unifiés (ColPali, ImageBind) pour indexer et récupérer des documents complexes sans conversion préalable.

Mots-clés

RAG, ColPali, Multimodal embeddings, CLIP, Document AI, ImageBind, Visual RAG, Heterogeneous retrieval

Abstract

Retrieval-Augmented Generation (RAG) has become the de facto approach for grounding large language model responses in enterprise knowledge bases. However, existing RAG systems are predominantly text-centric and fail on real-world document corpora that intermix natural language, structured tables, technical diagrams, and visual annotations. This paper presents MultiRAG, a modular pipeline for multimodal document retrieval that bypasses traditional OCR by operating directly on document page images using vision-language embedding models. We systematically compare four embedding strategies — ColPali patch-level embeddings, CLIP-based page representations, late interaction models, and hybrid text-visual fusion — across three industrial document datasets: engineering manuals (3,400 pages), medical imaging reports (1,800 pages), and legal contracts with embedded charts (2,100 pages). MultiRAG achieves a 34% improvement in Recall@5 over text-only RAG on diagram-heavy documents and reduces hallucination rate by 22% on questions requiring visual context. We further analyze computational trade-offs and provide deployment guidelines for production RAG systems handling heterogeneous document types.

Sujet 5 — Architecture complète d'un agent IA de production : perception, planification, action et auto-réflexion

Problématique

Malgré la prolifération des agents IA en recherche, il n'existe pas de consensus sur une architecture de référence pour des agents déployables en production. Les travaux récents (ReAct, Reflexion, LATS) proposent des modules partiels sans offrir une vision systémique intégrant la gestion des erreurs, la supervision humaine, la traçabilité des décisions et les contraintes de sécurité. Cette lacune génère des agents fragiles, imprévisibles et difficiles à auditer. Cette recherche définit et implémente une architecture de référence complète pour agents IA de production, validée sur des cas d'usage industriels réels.

Mots-clés

LATS, Reflexion, ReAct, Agent architecture, Self-critique, Human-in-the-loop, Production AI, Decision traceability

Abstract

The deployment of agentic AI systems in production environments demands architectural rigor that goes beyond academic prototypes. While frameworks like ReAct, Reflexion, and LATS have advanced reasoning and self-correction capabilities, they lack the cross-cutting concerns essential for reliable production systems: structured error recovery, human oversight integration, decision auditability, and graceful degradation. This paper presents ProdAgent, a reference architecture for production-grade agentic AI systems organized around five interacting subsystems: a multimodal perception layer, a hierarchical planning engine with constraint checking, an action execution module with sandboxed tool invocation, a reflective self-evaluation loop, and a human oversight interface with configurable intervention thresholds. We formalize the architecture using a component interaction model and provide implementation guidelines using open-source components. Validation across three enterprise use cases — automated customer support triage, code review automation, and supply chain anomaly investigation — demonstrates that ProdAgent achieves 91% task completion with a mean-time-to-failure 4.2× longer than baseline ReAct implementations. We release the architecture specification, reference implementation, and evaluation datasets as open-source artifacts.

Sujet 6 — Agent-to-Agent (A2A) : protocoles de communication sécurisée entre agents IA hétérogènes

Problématique

Le déploiement de systèmes multi-agents nécessite que des agents développés par différentes équipes, potentiellement avec des LLM différents, puissent collaborer de manière fiable et sécurisée. Le protocole A2A de Google (2025) propose un standard de communication inter-agents via des AgentCards et des Task objects, mais soulève des questions critiques de sécurité (usurpation d'identité d'agent, manipulation de tâches), d'interopérabilité avec MCP, et de gestion du consensus dans des environnements adversariaux. Cette recherche analyse formellement A2A, propose un modèle de menaces exhaustif et conçoit des mécanismes d'attestation cryptographique pour sécuriser les échanges inter-agents.

Mots-clés

A2A protocol, Google, Multi-agent systems, Inter-agent trust, AgentCard, Cryptographic attestation, Agent identity, AAMAS

Abstract

The emergence of large-scale multi-agent AI systems necessitates robust protocols for inter-agent communication that address identity verification, task integrity, and capability negotiation. Google's Agent-to-Agent (A2A) protocol, released in 2025, provides a promising open standard for agent interoperability through AgentCards, Task objects, and a defined message lifecycle. However, its security model has not been formally analyzed, and its interaction with the Model Context Protocol (MCP) in hybrid deployments remains underspecified. This paper presents a comprehensive security analysis of A2A through formal threat modeling, identifying six attack classes including agent impersonation, task hijacking, and capability inflation attacks. We propose A2A-Secure, an extension that introduces EdDSA-based AgentCard signing, capability attestation anchored to verifiable credentials, and a lightweight consensus mechanism for multi-agent task delegation chains. We implement and evaluate A2A-Secure on a testbed of 12 heterogeneous agents (GPT-4o, Claude 3.5, Llama 3.1) executing collaborative software development workflows. Results demonstrate that A2A-Secure prevents all identified attack vectors with a communication overhead of less than 8%, making it viable for production multi-agent deployments.

Sujet 7 — Orchestrateurs d'agents IA : comparaison LangGraph, Temporal, Prefect et approches LLM-native

Problématique

L'orchestration de workflows agentiques complexes — impliquant des boucles conditionnelles, des exécutions parallèles, des reprises sur erreur et des états persistants — représente un défi d'ingénierie non résolu. Les orchestrateurs classiques (Temporal, Prefect, Airflow) ne sont pas conçus pour la nature probabiliste des LLM, tandis que les orchestrateurs LLM-native (LangGraph) manquent de maturité opérationnelle. Cette recherche propose un cadre de sélection fondé sur des benchmarks de résilience, de cohérence et de performance sur des workflows agentiques de complexité variable.

Mots-clés

LangGraph, Orchestration, DAG agents, Workflow reliability, Prefect, Temporal, Airflow, Agentic workflow

Abstract

Orchestrating complex agentic AI workflows — characterized by conditional branching, parallel tool execution, stateful long-running processes, and stochastic LLM behavior — presents unique engineering challenges that neither traditional workflow orchestrators nor nascent LLM-native frameworks fully address. This paper presents a comparative study of five orchestration approaches: LangGraph, Temporal, Prefect, Apache Airflow with LLM operators, and a custom LLM-native event-driven architecture. We design an agentic workflow benchmark suite with 45 scenarios across three complexity tiers, evaluating orchestrators on six dimensions: task completion rate, fault tolerance, latency, resource efficiency, developer experience, and operational observability. Our experiments reveal a fundamental tension between workflow determinism (favoring Temporal) and reasoning flexibility (favoring LangGraph), with no single orchestrator dominating across all dimensions. We introduce a decision framework mapping workflow characteristics to orchestrator selection criteria and demonstrate how a hybrid architecture combining LangGraph for reasoning cycles with Temporal for durability achieves state-of-the-art results. Implications for production agentic system design are discussed.

Sujet 8 — Raisonnement étendu dans les LLM : analyse de l'inférence à temps de calcul variable (o1, o3, Gemini Thinking)

Problématique

Une nouvelle génération de LLM (OpenAI o1/o3, Gemini 2.0 Thinking, DeepSeek R1) exploite un raisonnement étendu à l'inférence — le test-time compute — pour améliorer les performances sur des tâches complexes nécessitant plusieurs étapes de déduction. Bien que ces modèles affichent des gains impressionnants sur des benchmarks mathématiques et scientifiques, les mécanismes sous-jacents, les limites et les compromis coût/performance restent mal compris. Cette recherche analyse systématiquement les gains apportés par le raisonnement étendu selon la catégorie de tâche, la longueur de chaîne de pensée et le budget computationnel alloué.

Mots-clés

Chain-of-Thought, o3, Test-time compute, Extended reasoning, DeepSeek R1, MATH benchmark, Reasoning LLM, Inference scaling

Abstract

Test-time compute scaling — the allocation of additional computational resources to generate extended reasoning chains during inference — has emerged as a powerful paradigm for improving LLM performance on complex tasks. Models such as OpenAI o3, Google Gemini 2.0 Flash Thinking, and DeepSeek R1 demonstrate significant gains on mathematical reasoning, scientific problem-solving, and multi-step planning benchmarks by generating lengthy chain-of-thought traces before producing final answers. This paper presents a systematic analysis of test-time compute benefits across 12 task categories, 5 reasoning budget levels, and 6 state-of-the-art reasoning models. We introduce a reasoning efficiency metric (MATH score per 1000 tokens) that reveals non-linear returns: extended reasoning provides substantial gains for problems requiring 3–7 deductive steps but diminishing returns beyond this range. We further analyze the qualitative structure of extended reasoning traces, identifying five reasoning strategies (decomposition, analogy, verification, backtracking, self-consistency voting) and their task-specific effectiveness. Our findings provide actionable guidance for deploying reasoning-capable LLMs in cost-sensitive production environments.

Sujet 9 — Sécurité des agents IA autonomes : injection de prompt indirecte, sandboxing et défenses systémiques

Problématique

Les agents IA autonomes, qui exécutent des outils et interagissent avec des environnements externes, introduisent une surface d'attaque qualitativement différente des LLM statiques. L'injection de prompt indirecte — où du contenu malveillant dans l'environnement manipule l'agent — représente une menace critique insuffisamment étudiée. Par ailleurs, les mécanismes de sandboxing et de guardrails existants ne sont pas conçus pour les propriétés émergentes des agents (planification autonome, enchaînement d'actions). Cette recherche cartographie exhaustivement les vecteurs d'attaque spécifiques aux agents et propose un framework de défense à plusieurs niveaux.

Mots-clés

Prompt injection, Sandboxing, Guardrails, Red teaming, Agent security, Indirect injection, Constitutional AI, Agentic threat model

Abstract

Autonomous AI agents that execute multi-step plans, invoke external tools, and process environmental feedback introduce security vulnerabilities fundamentally distinct from those of static LLMs. Indirect prompt injection — where adversarial content embedded in tool outputs, web pages, or documents manipulates agent behavior — has emerged as a critical yet understudied attack vector. This paper presents AgentThreat, a comprehensive threat modeling framework for autonomous AI agents, enumerating 23 attack patterns across four categories: input manipulation, tool abuse, state corruption, and goal hijacking. We empirically evaluate these attacks against five production agent frameworks, achieving a mean attack success rate of 67% with unsophisticated adversarial payloads. In response, we propose DefendAgent, a multi-layer defense architecture combining instruction-following firewalls, sandboxed tool execution with capability whitelisting, behavioral anomaly detection, and cryptographic provenance tracking for action chains. DefendAgent reduces attack success rates to below 12% with a 15% computational overhead, establishing a practical security baseline for production agentic deployments. We release our attack corpus and defense implementation to support reproducible security research.

Sujet 10 — Fine-tuning efficace de LLM pour domaines spécialisés : LoRA, QLoRA et adapter fusion

Problématique

L'adaptation de LLM de grande taille à des domaines spécialisés (médecine, droit, ingénierie, langues à faible ressource) nécessite traditionnellement un fine-tuning complet coûteux et sujet à l'oubli catastrophique. Les méthodes PEFT (Parameter-Efficient Fine-Tuning), notamment LoRA et QLoRA, permettent une adaptation efficace mais leurs hyperparamètres optimaux varient significativement selon le domaine, la langue et l'architecture du modèle de base. Cette recherche conduit une étude systématique des facteurs influençant l'efficacité du fine-tuning PEFT sur des corpus techniques arabes et francophones en ingénierie.

Mots-clés

LoRA, QLoRA, PEFT, Domain adaptation, Catastrophic forgetting, Arabic NLP, Low-resource, Adapter fusion

Abstract

Parameter-efficient fine-tuning (PEFT) methods have democratized the adaptation of large language models to specialized domains by enabling high-quality fine-tuning with a fraction of the computational resources required for full fine-tuning. However, systematic understanding of how PEFT hyperparameters — rank, alpha, target modules, learning rate — interact with domain characteristics, language typology, and base model architecture remains limited. This paper presents a large-scale empirical study of LoRA, QLoRA, DoRA, and AdaLoRA on five specialized Arabic and French engineering corpora: hydraulic engineering technical manuals, industrial maintenance reports, academic engineering papers, patent documents, and regulatory standards. We evaluate 180 fine-tuning configurations across three base models (Mistral-7B, Qwen-2.5-7B, LLaMA-3.1-8B) using domain-specific benchmarks for technical question answering, terminology extraction, and document summarization. Our results identify optimal PEFT configurations for each domain-language combination and quantify the trade-off between parameter efficiency and downstream task performance. We further introduce a domain complexity score that predicts the minimum rank required to achieve within 95% of full fine-tuning performance, providing practitioners with principled configuration guidelines.

Sujet 11 — GraphRAG et récupération hybride : graphes de connaissance et vecteurs pour le raisonnement multi-hop

Problématique

Les systèmes RAG denses classiques excellent dans la récupération de passages individuellement pertinents mais échouent sur des questions nécessitant un raisonnement relationnel multi-hop — c'est-à-dire l'agrégation d'informations dispersées dans plusieurs documents liés par des relations sémantiques. GraphRAG (Microsoft, 2024) propose d'indexer les documents sous forme de graphes de communautés pour améliorer le raisonnement global, mais ses coûts d'indexation élevés et ses performances variables sur des corpus spécialisés limitent son adoption. Cette recherche développe une stratégie de récupération hybride adaptative combinant RAG dense et GraphRAG selon la complexité de la requête.

Mots-clés

GraphRAG, Neo4j, Multi-hop QA, Hybrid retrieval, Knowledge graph, HippoRAG, Community detection, Entity resolution

Abstract

Complex question answering over enterprise knowledge bases frequently requires multi-hop reasoning — the ability to synthesize information distributed across multiple documents connected through implicit relational chains. Standard dense RAG systems, optimized for single-document relevance, systematically fail on such queries. Microsoft's GraphRAG framework addresses this by constructing hierarchical community graphs from document corpora, enabling global reasoning over entity relationships. However, GraphRAG's high indexing costs (5–8× dense RAG) and inconsistent performance on specialized technical corpora limit its practical applicability. This paper introduces AdaptRAG, a query-complexity-aware retrieval framework that dynamically routes queries between dense vector retrieval, entity-centric graph traversal, and full GraphRAG community summarization based on structural query analysis. We evaluate AdaptRAG on three multi-hop benchmarks — HotpotQA, 2WikiMultiHopQA, and a novel engineering standards dataset — demonstrating a 31% improvement in F1 over GraphRAG on single-hop queries and a 24% improvement over dense RAG on multi-hop queries, with 40% lower average indexing cost. Our routing mechanism achieves 89% accuracy in query complexity classification using a lightweight 7B discriminator model.

Sujet 12 — LLM vision-langage pour l'analyse automatique de documents techniques et schémas d'ingénierie

Problématique

L'analyse automatique de documents d'ingénierie — plans architecturaux, schémas électriques, diagrammes hydrauliques, P&ID — représente un défi majeur pour les systèmes de compréhension documentaire. Ces documents combinent symboles normalisés, annotations textuelles, structures spatiales et conventions sectorielles que les OCR classiques ne peuvent traiter. Les LLM vision-langage récents (GPT-4V, Gemini Pro Vision, LLaVA, InternVL) offrent une capacité de raisonnement visuel prometteuse, mais leurs performances sur des documents techniques industriels restent non évaluées de manière rigoureuse. Cette recherche crée un benchmark spécialisé et évalue systématiquement ces modèles.

Mots-clés

Vision LLM, Document AI, LLaVA, GPT-4V, InternVL, Engineering diagrams, P&ID, Technical document understanding

Abstract

Technical engineering documents — including P&ID diagrams, electrical schematics, architectural blueprints, and hydraulic circuit drawings — represent a critical class of industrial knowledge that remains largely inaccessible to automated AI analysis. These documents feature domain-specific symbolic vocabularies, spatial reasoning requirements, and standardized notation systems (ISO, IEC, ANSI) that pose fundamental challenges for both OCR-based and purely language-oriented approaches. This paper presents EngDoc-Bench, a curated benchmark of 2,400 annotated technical documents across six engineering domains, supporting four evaluation tasks: symbol recognition, component relationship extraction, specification question answering, and anomaly detection. We conduct a systematic evaluation of eight vision-language models — GPT-4o, Gemini 1.5 Pro, LLaVA-1.6, InternVL-2, Qwen-VL-Max, Claude 3.5 Sonnet, Phi-3.5-Vision, and Idefics3 — revealing substantial performance variation across document types. GPT-4o achieves the highest overall performance (67.3% accuracy on Q&A) but fails systematically on densely annotated P&ID diagrams. We further show that fine-tuning LLaVA-1.6 on 800 domain-specific examples yields competitive performance (64.1%) at significantly lower inference cost, providing a practical deployment pathway for industry.

Sujet 13 — Compression et déploiement d'agents IA sur systèmes embarqués : quantisation, pruning et distillation

Problématique

Le déploiement d'agents IA intelligents dans des contextes embarqués et edge computing (drones, robots industriels, véhicules autonomes, équipements médicaux) est contraint par des budgets énergétiques stricts ($< 15W$) et des limitations mémoire ($< 8 Go$ RAM) incompatibles avec les LLM de grande taille. Si les techniques de compression (quantisation INT4/INT8, pruning structuré, distillation de connaissance) permettent de réduire significativement les modèles, leur impact sur les capacités de raisonnement et d'appel d'outils — essentielles aux agents — reste peu étudié. Cette recherche évalue systématiquement ces compromis et propose un pipeline de compression optimisé pour agents embarqués.

Mots-clés

Quantisation, Edge AI, ONNX, llama.cpp, Knowledge distillation, Structured pruning, Embedded LLM, Jetson Orin

Abstract

Deploying agentic AI capabilities on resource-constrained edge devices — industrial robots, UAVs, medical equipment, and autonomous vehicles — requires bridging a significant gap between the computational demands of modern LLMs and the strict power and memory budgets of embedded systems. While compression techniques such as INT4 quantization, structured pruning, and knowledge distillation have demonstrated impressive model size reductions, their impact on the specific capabilities required for agentic behavior — multi-step reasoning, tool invocation accuracy, context maintenance, and self-correction — has not been systematically characterized. This paper presents EdgeAgent, a compression-aware framework for deploying capable AI agents on NVIDIA Jetson Orin (15W) and Raspberry Pi 5 (8W) platforms. We evaluate 24 compression configurations of Mistral-7B, Phi-3.5-Mini, and Qwen-2.5-3B across five agentic benchmarks, measuring the compression-capability Pareto frontier. Our results demonstrate that a 4-bit quantized Phi-3.5-Mini achieves 78% of the tool-calling accuracy of its full-precision counterpart at 4.2× lower memory footprint, while structured pruning introduces disproportionate degradation in multi-step reasoning. We further introduce an agent-aware distillation procedure that transfers agentic behavioral patterns rather than token distributions, achieving a 12% improvement over standard distillation on tool-use benchmarks.

Sujet 14 — Agents IA pour l'ingénierie logicielle autonome : révision de code, détection de vulnérabilités et correction automatique

Problématique

L'intégration d'agents IA dans les pipelines de développement logiciel — pour la revue de code, la détection de bugs, la correction automatique et la génération de tests — soulève des questions fondamentales sur la fiabilité, la sécurité et l'alignement des modifications proposées. Les agents de codage actuels (Devin, GitHub Copilot Workspace, SWE-agent) peinent à traiter des bases de code larges et complexes, à maintenir la cohérence architecturale et à éviter l'introduction de nouvelles vulnérabilités lors des corrections. Cette recherche propose une architecture d'agent de révision de code orientée sécurité et évalue ses performances sur des projets open-source réels.

Mots-clés

SWE-bench, Devin, CodeAct, Code agent, Software engineering, Vulnerability detection, Automated repair, GitHub Copilot

Abstract

Autonomous AI agents capable of understanding, reviewing, and repairing production code represent a transformative opportunity for software engineering productivity and security. Current coding agents — including Devin, SWE-agent, and GitHub Copilot Workspace — demonstrate impressive capabilities on isolated programming tasks but exhibit systematic weaknesses on large codebases: they fail to maintain architectural consistency across modifications, introduce new security vulnerabilities while fixing targeted bugs, and lack effective mechanisms for reasoning about cross-file dependencies. This paper presents SecureCodeAgent, an agent architecture specifically designed for security-aware code review and automated vulnerability remediation. The agent combines a hierarchical codebase indexing strategy with a security-specialized reasoning module trained on 50,000 CVE-annotated code samples. We evaluate SecureCodeAgent on SWE-bench (verified subset), a novel security vulnerability repair benchmark (SecVulnBench-500) covering OWASP Top 10 categories, and five real-world open-source repositories. SecureCodeAgent achieves 38.2% resolve rate on SWE-bench (state-of-the-art for open-weight models) while reducing introduced vulnerability rate to 2.1% compared to 8.7% for SWE-agent. Our analysis reveals that security-aware training reduces repair quality only marginally while substantially improving safety of generated patches.

Sujet 15 — Détection et mitigation des hallucinations dans les pipelines agentiques : calibration, self-consistency et factualité

Problématique

Les hallucinations des LLM — la génération de faits plausibles mais inexacts — représentent un risque critique dans les agents IA déployés dans des contextes à forts enjeux (médical, juridique, industriel). Dans les pipelines agentiques, les hallucinations se propagent et s'amplifient à travers les chaînes d'actions, transformant des erreurs ponctuelles en séquences de décisions incorrectes. Les méthodes de détection existantes (self-consistency, conformal prediction, factuality scoring) ont été développées pour des LLM statiques et ne s'adaptent pas aux propriétés dynamiques des agents. Cette recherche conçoit un module anti-hallucination plug-and-play pour pipelines agentiques RAG.

Mots-clés

Hallucination detection, Self-consistency, Factuality scoring, Conformal prediction, TruthfulQA, Agentic reliability, RAGAS, Uncertainty quantification

Abstract

Hallucinations — the generation of factually incorrect yet linguistically plausible content — pose a particularly severe risk in agentic AI pipelines, where LLM outputs directly drive tool invocations, data modifications, and multi-step decision chains. Unlike static LLM deployments, agentic settings amplify hallucination impact through error propagation: a single fabricated fact in step 2 of a 10-step pipeline can corrupt all subsequent reasoning and actions. Existing hallucination mitigation approaches, including self-consistency voting, conformal prediction intervals, and retrieval-based factuality scoring, were designed for isolated inference calls and do not account for sequential state dependencies in agentic contexts. This paper introduces HalluGuard, a plug-and-play hallucination detection and mitigation module designed for integration into agentic RAG pipelines. HalluGuard operates through three complementary mechanisms: cross-source consistency verification (detecting claim disagreements across retrieved documents), temporal coherence checking (flagging contradictions with previously established facts in the agent's context), and calibrated confidence estimation using a lightweight verifier model trained on 200,000 human-annotated claim-evidence pairs. Evaluation on TruthfulQA-Agent, HaluEval-Agentic (our novel benchmark of 3,000 multi-step agentic scenarios), and three production RAG deployments demonstrates that HalluGuard reduces hallucination rate by 54% with a 9% increase in inference latency, achieving a favorable Pareto trade-off for safety-critical applications.